

CSCE 775: Deep Reinforcement Learning and Search

A Comprehensive Survey of Reinforcement Learning Methods for Large Language Model Alignment

JC Vaught

March 2, 2026

Contents

1	Introduction	4
2	The RLHF Pipeline	4
2.1	Stage 1: Supervised Fine-Tuning	4
2.2	Stage 2: Reward Model Training	5
2.3	Stage 3: Policy Optimization	5
3	Policy Gradient Methods	5
3.1	Proximal Policy Optimization (PPO)	5
3.1.1	Architecture	5
3.1.2	Advantage Estimation	6
3.1.3	Clipped Surrogate Objective	6
3.1.4	Adoption	6
3.2	GRPO (Group Relative Policy Optimization)	6
3.2.1	Group Sampling and Advantage Computation	7
3.2.2	Objective Function	7
3.2.3	KL Divergence Estimator	7
3.2.4	Key Properties	7
3.3	REINFORCE++	8
3.3.1	Global Batch Normalization	8
3.3.2	Token-Level KL Penalty	8

3.3.3	Training Efficiency	8
3.4	CISPO (Clipped Importance Sampling Policy Optimization)	9
3.5	GSPO (Group Sequence Policy Optimization)	9
3.6	Other Policy Gradient Variants	9
4	Direct Preference Optimization Family	10
4.1	DPO (Direct Preference Optimization)	10
4.2	IPO (Identity Preference Optimization)	10
4.3	KTO (Kahneman-Tversky Optimization)	11
4.4	ORPO (Odds Ratio Preference Optimization)	11
4.5	SimPO (Simple Preference Optimization)	12
5	Rejection Sampling and Best-of-N Methods	12
5.1	Best-of-N Sampling	12
5.2	RAFT (Reward-Ranked Fine-Tuning)	12
5.3	RSO (Statistical Rejection Sampling Optimization)	12
6	AI Feedback and Verifiable Reward Methods	13
6.1	RLAIF (Reinforcement Learning from AI Feedback)	13
6.2	Constitutional AI (CAI)	13
6.3	RLVR (Reinforcement Learning from Verifiable Rewards)	13
6.4	SteerLM	13
7	Hybrid and Multi-Stage Pipelines	14
7.1	The Emerging Standard Pipeline	14
7.2	Alternative Architectures	14
8	Industry Adoption	15
9	Comparative Analysis of Advantage Mechanisms	15
10	Security and Robustness	15

10.1	Data Poisoning Attacks on Preference Data	15
10.1.1	Attack Methodology	15
10.1.2	Known Vulnerability Thresholds	16
10.1.3	GRPO and REINFORCE++: Theoretical Analysis	16
10.2	Reward Hacking and Emergent Misalignment	16
10.3	Benchmarks and Attack Frameworks	17
10.4	Defense Mechanisms	17
11	Evaluation Methodology	18
11.1	Safety Classifiers	18
11.2	General Capability Benchmarks	18
12	Open Questions and Future Directions	18
13	Conclusion	19

1. Introduction

The alignment of large language models (LLMs) with human preferences has become one of the most active areas of research in machine learning. Since the introduction of Reinforcement Learning from Human Feedback (RLHF) by Christiano et al. [12] and its successful application in InstructGPT [40], the field has produced a rapidly growing family of alignment algorithms. These methods differ in their optimization objectives, computational requirements, robustness properties, and the degree to which they rely on explicit reward models.

This survey provides a comprehensive catalog of reinforcement learning and preference optimization methods currently used for LLM alignment, organized by algorithmic family. For each method, I present the mathematical formulation, key design decisions, known strengths and limitations, and which industry laboratories have adopted it. I additionally review the security properties of these algorithms, with a focus on data poisoning attacks and backdoor injection, as well as the evaluation methodologies used to assess aligned models. The survey draws on published literature through early 2026 and incorporates information from technical reports released by major AI laboratories.

The scope encompasses four broad families of methods. *Policy gradient methods* such as PPO [46], GRPO [48], and REINFORCE++ [28] optimize a language model policy against a learned or verifiable reward signal using gradient-based reinforcement learning. *Direct preference optimization methods* such as DPO [44], IPO [3], KTO [16], ORPO [23], and SimPO [34] bypass the reward model entirely and learn directly from preference pairs in a supervised fashion. *Rejection sampling methods* generate multiple candidate responses and fine-tune on the best ones as scored by a reward model [51, 14]. Finally, *AI feedback methods* replace human annotations with model-generated evaluations, including RLAIIF [31], Constitutional AI [5], and Reinforcement Learning from Verifiable Rewards (RLVR) [30].

2. The RLHF Pipeline

Before examining individual algorithms, it is helpful to understand the standard RLHF pipeline from which most methods derive or depart. The canonical pipeline, as described in InstructGPT [40], consists of three stages.

2.1. Stage 1: Supervised Fine-Tuning

A pretrained language model is fine-tuned on a dataset of high-quality demonstrations, producing an SFT model π_{SFT} . This model serves as both the starting policy and the reference policy π_{ref} for subsequent RL training.

2.2. Stage 2: Reward Model Training

A reward model $r_\phi(x, y)$ is trained on human preference data. Given a prompt x and two responses y_w (preferred) and y_l (rejected), the reward model is optimized under the Bradley-Terry preference model.

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))] \quad (1)$$

2.3. Stage 3: Policy Optimization

The SFT model is further trained to maximize the reward model’s output while staying close to the reference policy via a KL penalty.

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [r_\phi(x, y) - \beta D_{\text{KL}}[\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)]] \quad (2)$$

The parameter β controls the strength of the KL constraint, preventing the policy from diverging too far from the reference and exploiting reward model artifacts (reward hacking). Different alignment algorithms implement this third stage in fundamentally different ways, and the choice of algorithm affects computational cost, training stability, and robustness to adversarial perturbations. The following sections detail each family of methods.

3. Policy Gradient Methods

Policy gradient methods directly optimize the language model policy π_θ using gradient estimates derived from sampled trajectories (generated text sequences) and their associated rewards. These methods differ primarily in how they estimate the *advantage* $\hat{A}$, which measures how much better a particular action (token generation) is compared to the expected value under the current policy.

3.1. Proximal Policy Optimization (PPO)

PPO [46] is the original algorithm used in InstructGPT [40] for the policy optimization stage of RLHF. It remains the most thoroughly studied alignment algorithm and the baseline against which newer methods are evaluated.

3.1.1. Architecture

PPO requires four models in memory simultaneously during training. The *policy model* π_θ is the language model being optimized. The *reference model* π_{ref} is a frozen copy of the SFT model, used to compute the KL penalty. The *reward model* r_ϕ scores generated responses. The *critic (value) model* $V_\psi(s)$ estimates the expected return from each state s and is used to compute advantage estimates.

3.1.2. Advantage Estimation

PPO uses Generalized Advantage Estimation (GAE) to compute advantages. The temporal difference (TD) residual at token position t is

$$\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t) \quad (3)$$

where r_t is the reward at step t (typically zero except at the final token, where the reward model score is applied) and γ is the discount factor. The GAE advantage is

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{T-t-1} (\gamma \lambda)^k \delta_{t+k} \quad (4)$$

where $\lambda \in [0, 1]$ controls the bias-variance tradeoff. Typical values for RLHF are $\lambda = 0.95$ and $\gamma \approx 1.0$.

3.1.3. Clipped Surrogate Objective

The policy is updated by maximizing a clipped surrogate objective that prevents excessively large policy updates.

$$\mathcal{L}_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t, \text{clip} \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right] \quad (5)$$

The clipping parameter ϵ (typically 0.2) ensures that the probability ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$ does not deviate too far from 1.0, producing conservative policy updates. The combined PPO loss also includes a value function loss $\mathcal{L}_{\text{VF}}$ and an entropy bonus $\mathcal{L}_{\text{ent}}$ that encourages exploration.

3.1.4. Adoption

PPO was the primary alignment algorithm used by OpenAI for ChatGPT and GPT-4, by Anthropic in earlier versions of their models, and remains widely used in RLHF research. Its main drawback is computational cost, as maintaining four models in memory and performing rollout generation, advantage computation, and policy updates requires substantial GPU resources.

3.2. GRPO (Group Relative Policy Optimization)

GRPO [48] was introduced in DeepSeek-Math and subsequently used to train DeepSeek-R1 [20], making it one of the most consequential alignment algorithms in production. Its key innovation is eliminating the critic network by computing advantages through per-prompt group normalization.

3.2.1. Group Sampling and Advantage Computation

For each prompt q , GRPO samples a group of G responses $\{o_1, o_2, \dots, o_G\}$ from the current policy $\pi_{\theta_{\text{old}}}$. Each response receives a scalar reward r_i from the reward model. The advantage for response i is computed by normalizing rewards within the group.

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^G)}{\text{std}(\{r_j\}_{j=1}^G) + \varepsilon} \quad (6)$$

This normalized advantage is applied uniformly to all tokens in response i , so that $\hat{A}_{i,t} = \hat{A}_i$ for all token positions t .

3.2.2. Objective Function

GRPO optimizes a clipped surrogate objective averaged over the group, with an explicit KL penalty.

$$J_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left[\min\left(\rho_{i,t} \hat{A}_{i,t}, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon) \hat{A}_{i,t}\right) - \beta D_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right] \quad (7)$$

where $\rho_{i,t} = \pi_{\theta}(o_{i,t}|q, o_{i,<t})/\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})$ is the importance sampling ratio.

3.2.3. KL Divergence Estimator

GRPO uses the k_3 KL estimator.

$$D_{\text{KL}}^{k_3} = \delta(y) - 1 - \log \delta(y), \quad \text{where} \quad \delta = \frac{\pi_{\text{ref}}}{\pi_{\theta}} \quad (8)$$

The REINFORCE++ paper [28] identifies this estimator as suffering from high variance and numerical instability, requiring frequent policy resets to prevent divergence during training.

3.2.4. Key Properties

GRPO reduces memory from four models (PPO) to three models (policy, reference, reward) by eliminating the critic. Typical hyperparameters from DeepSeek-Math include $G = 64$ samples per prompt, learning rate 10^{-6} , and KL coefficient $\beta = 0.04$. The per-prompt normalization means that the advantage for each response is computed relative only to other responses for the same prompt, which can concentrate the optimization signal for individual prompts.

3.3. REINFORCE++

REINFORCE++ [28] was introduced as a simpler alternative to both PPO and GRPO. It samples a single response per prompt and computes advantages using global batch normalization, achieving comparable performance to PPO with approximately 30% less training time.

3.3.1. Global Batch Normalization

Unlike GRPO’s per-prompt normalization, REINFORCE++ normalizes advantages across the entire training batch.

$$\hat{A}_{q,o_t}^{\text{norm}} = \frac{\hat{A}_{q,o_t} - \text{mean}(\hat{A} \mid \hat{A} \in \mathcal{D}_{\text{batch}})}{\text{std}(\hat{A} \mid \mathcal{D}_{\text{batch}}) + \varepsilon} \quad (9)$$

For the baseline variant (sampling $k > 1$ responses per prompt), a two-step process is used. First, the group mean is subtracted within each prompt. Second, the result is normalized globally across the batch.

3.3.2. Token-Level KL Penalty

REINFORCE++ uses the k_1 KL estimator, embedded directly into the advantage at the token level.

$$\text{KL}(t) = \log \frac{\pi_{\theta_{\text{old}}}^{\text{RL}}(o_t \mid q, o_{<t})}{\pi^{\text{SFT}}(o_t \mid q, o_{<t})} \quad (10)$$

The advantage at token t incorporates cumulative future KL.

$$A_{q,o_t} = r(o_{1:T}, q) - \beta \sum_{i=t}^T \text{KL}(i) \quad (11)$$

This token-level formulation provides finer-grained control than sequence-level KL penalties, with the KL contribution naturally decaying from earlier tokens to later tokens. The baseline variant uses the k_2 estimator $D_{\text{KL}}^{k_2} = \frac{1}{2}(\log(\pi_{\text{ref}}/\pi_{\theta}))^2$, which has been shown to provide more stable, unbiased gradient estimation than GRPO’s k_3 estimator.

3.3.3. Training Efficiency

On Llama-3-8B with 70K training samples on H100 GPUs, REINFORCE++ reduced RLHF training time from 60 hours (PPO) to 42 hours. Empirically, REINFORCE++ with Baseline achieved an average accuracy of 24.10 across standard benchmarks, outperforming GRPO (22.58) and PPO (21.85) [28].

3.4. CISPO (Clipped Importance Sampling Policy Optimization)

CISPO is MiniMax’s proprietary RL algorithm, used to train the MiniMax-M1 and MiniMax-M2.5 model families [36]. Unlike PPO, which clips the policy ratio, CISPO clips the importance sampling weights directly. This design choice provides superior training stability for MiniMax’s hybrid attention architectures (combining standard attention with lightning attention) and mixture-of-experts models. MiniMax reports that CISPO outperforms other competitive RL variants in their internal evaluations. Their Forge framework [35] extends CISPO with unified mixed-domain training, simultaneously training on reasoning, general QA, and agentic tasks rather than the sequential multi-stage RL pipelines used by most other laboratories.

3.5. GSPO (Group Sequence Policy Optimization)

GSPO is Alibaba’s successor to GRPO, developed specifically to address stability and scaling issues observed when training Qwen3’s large mixture-of-experts architecture [1]. The key innovation is a theoretically grounded importance ratio derived from sequence likelihood rather than token-level ratios, which aligns with importance sampling principles. Alibaba reports that GSPO significantly outperforms GRPO in stability, efficiency, and overall performance, particularly for large MoE models where GRPO exhibited training collapse. GSPO is implemented in Alibaba’s open-source ROLL framework, which also supports PPO, GRPO, REINFORCE++, and several other algorithms.

3.6. Other Policy Gradient Variants

Several additional policy gradient methods have been developed by specific laboratories.

DAPPO (Distributed Adaptive PPO) was developed by ByteDance/Seed for their veRL (VolcEngine RL) framework. It adapts the PPO clipping and learning rate parameters dynamically during training and was used to replicate DeepSeek’s R1-Zero results.

Online Mirror Descent was used by Moonshot AI for Kimi K1.5 [38], combined with partial rollouts that reuse large chunks of previous trajectories. They achieved strong performance without Monte Carlo tree search, value functions, or process reward models.

PARL (Parallel Agent Reinforcement Learning) was developed by Moonshot AI for Kimi K2.5, specifically designed to train models for parallel sub-agent decomposition. Early training rewards are shaped to encourage parallel execution and prevent “serial collapse” where the orchestrator defaults to running a single agent sequentially.

4. Direct Preference Optimization Family

Direct preference optimization methods bypass the reward model entirely, learning directly from preference pairs in a supervised fashion. This family has grown rapidly since the introduction of DPO in 2023, with each variant addressing specific limitations of the original formulation.

4.1. DPO (Direct Preference Optimization)

DPO [44] derives a closed-form solution for the optimal policy under the KL-constrained reward maximization objective (Equation 2). The optimal policy takes the form

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{r(x, y)}{\beta}\right) \quad (12)$$

Rearranging yields an implicit reward.

$$r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \quad (13)$$

Substituting into the Bradley-Terry preference model (the partition function $Z(x)$ cancels in the difference), the DPO loss becomes

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right] \right) \right] \quad (14)$$

The gradient of the DPO loss increases the probability of the preferred response and decreases the probability of the rejected response, weighted by the implicit DPO score $\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$. Samples where the model currently assigns higher implicit reward to the rejected response receive larger gradient updates.

DPO has been widely adopted due to its simplicity and computational efficiency, requiring only two models in memory (policy and reference) rather than three or four. Meta used DPO for alignment of Llama 2 and Llama 3 (in combination with rejection sampling) [51]. Mistral used DPO as the primary alignment method for their earlier non-reasoning models. However, DPO’s single-stage structure makes it vulnerable to data quality issues and, as discussed in Section 10, to data poisoning attacks.

4.2. IPO (Identity Preference Optimization)

IPO [3] addresses a theoretical issue with DPO’s loss function. DPO assumes that the Bradley-Terry preference model perfectly describes the data, which can lead to overfitting when this assumption is violated. IPO replaces the log-sigmoid loss with a squared hinge-like

loss that bounds the implicit reward difference.

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x, y_w, y_l)} \left[\left(\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} - \frac{1}{2\beta} \right)^2 \right] \quad (15)$$

This bounded loss function prevents the implicit reward from growing without bound, providing better regularization. PoisonBench [18] found that IPO is the most resilient algorithm among the DPO family to data poisoning, with an average attack success rate approximately 7 percentage points lower than standard DPO.

4.3. KTO (Kahneman-Tversky Optimization)

KTO [16] is motivated by prospect theory from behavioral economics, specifically the observation that humans are loss-averse and weight losses more heavily than equivalent gains. Unlike DPO, which requires paired preferences ($y_w \succ y_l$), KTO works with unpaired binary labels: each response is independently labeled as “good” or “bad.”

The KTO loss applies asymmetric weighting.

$$\mathcal{L}_{\text{KTO}}(\theta) = \mathbb{E}_{(x, y)} [w(y) \cdot (1 - v_{\theta}(x, y))] \quad (16)$$

where $v_{\theta}(x, y) = \sigma\left(\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} - z_{\text{ref}}\right)$ and the weight $w(y)$ is λ_w for desirable outputs and λ_l for undesirable ones, with $\lambda_l > \lambda_w$ reflecting loss aversion. The reference point z_{ref} is the expected reward under the reference policy.

KTO’s key advantage is data efficiency. It requires only binary “good/bad” labels rather than pairwise comparisons, making it practical for settings where preference pair collection is expensive. Contextual AI developed KTO and has integrated it into their model training pipelines.

4.4. ORPO (Odds Ratio Preference Optimization)

ORPO [23] combines supervised fine-tuning and preference alignment into a single training stage, eliminating the need for a separate SFT phase. The loss function adds an odds ratio penalty to the standard language modeling loss.

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{NLL}}(\theta) - \lambda \cdot \log \sigma \left(\log \frac{\text{odds}_{\theta}(y_w|x)}{\text{odds}_{\theta}(y_l|x)} \right) \quad (17)$$

where $\text{odds}_{\theta}(y|x) = \frac{P_{\theta}(y|x)}{1 - P_{\theta}(y|x)}$. By combining SFT and alignment, ORPO reduces the total number of training stages and eliminates the need for a reference model. The Zephyr model family (based on Mixtral) popularized ORPO in the open-source community.

4.5. SimPO (Simple Preference Optimization)

SimPO [34] simplifies DPO further by eliminating the reference model entirely. Instead of computing log-probability ratios between the policy and reference, SimPO uses the average log-probability of the response as an implicit reward.

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\frac{\beta}{|y_w|} \log \pi_{\theta}(y_w|x) - \frac{\beta}{|y_l|} \log \pi_{\theta}(y_l|x) - \gamma \right) \right] \quad (18)$$

The length normalization ($1/|y|$) addresses verbosity bias, and the margin term γ provides a target separation between preferred and rejected responses. SimPO requires only one model in memory during training, making it the most memory-efficient preference optimization method.

5. Rejection Sampling and Best-of-N Methods

Rejection sampling methods take a fundamentally different approach. Rather than optimizing the policy through gradient-based RL, they generate multiple candidate responses, score them with a reward model, and fine-tune on the highest-scoring outputs.

5.1. Best-of-N Sampling

The simplest form generates N responses for each prompt, selects the best one according to the reward model, and uses these selected responses as supervised fine-tuning data. Meta used rejection sampling extensively for Llama 2 [51], typically with best-of-16 selection against an early preference model. The method is conceptually straightforward and avoids the training instabilities associated with online RL, but it requires generating N complete responses for every training prompt, which is computationally expensive at large scale.

5.2. RAFT (Reward-Ranked Fine-Tuning)

RAFT [14] formalizes rejection sampling fine-tuning with a more principled selection mechanism. Rather than simply selecting the top-1 response, RAFT uses a reward-ranked threshold to select a subset of high-quality responses, providing more training signal per prompt. The method can be iterated, with the reward threshold adjusted as the policy improves.

5.3. RSO (Statistical Rejection Sampling Optimization)

RSO improves the statistical efficiency of rejection sampling by better approximating the optimal policy distribution. Standard rejection sampling from the SFT policy is inefficient because the SFT policy may assign low probability to high-reward responses. RSO uses importance weighting to correct for this distribution mismatch.

6. AI Feedback and Verifiable Reward Methods

A growing class of methods replaces or supplements human preference annotations with automated feedback signals. These approaches address the cost and scalability limitations of human annotation while enabling training on domains where human evaluation is difficult.

6.1. RLAIIF (Reinforcement Learning from AI Feedback)

RLAIIF [31] replaces human annotators with a capable language model that generates preference labels. A stronger LLM (the “judge”) evaluates pairs of responses and produces preference rankings, which are then used to train a reward model following the standard RLHF pipeline. Google has used RLAIIF extensively for Gemini, and the approach has become standard practice at most frontier laboratories as a cost-effective supplement to human annotation.

6.2. Constitutional AI (CAI)

Constitutional AI [5], developed by Anthropic, extends RLAIIF with a principled framework. The model first critiques its own outputs according to a set of written principles (the “constitution”), then revises the outputs to comply with those principles. The revised outputs are used as training data. This self-improvement loop enables alignment without relying on human preference labels for harmlessness, though human labels are still used for helpfulness. CAI was used for alignment of earlier Claude models.

6.3. RLVR (Reinforcement Learning from Verifiable Rewards)

RLVR, introduced by the Allen Institute for AI in Tülu 3 [30], trains language models using tasks with objectively checkable answers (e.g., mathematical correctness, code execution). Because the reward signal is deterministic and verifiable, RLVR avoids the noise and bias inherent in learned reward models. Tülu 3 demonstrated that RLVR alone can produce significant reasoning improvements without any human preference data. The approach has been adopted broadly: DeepSeek-R1 [20] uses GRPO with verifiable math rewards, and Mistral’s Magistral [37] uses GRPO with verifiable rewards from math and code correctness rather than learned reward models.

6.4. SteerLM

SteerLM, developed by NVIDIA [53], enables controllable generation by attaching multi-dimensional attribute labels to training data. Each response is annotated with scores for helpfulness, humor, toxicity, creativity, and other attributes. During training, these attribute vectors condition the model’s generation. At inference time, the user can “steer” generation

by specifying desired attribute values. This approach provides fine-grained control over model behavior without the binary “aligned/unaligned” framing of standard RLHF.

7. Hybrid and Multi-Stage Pipelines

In practice, most frontier laboratories use multi-stage pipelines that combine several of the above methods. The trend has converged toward a common recipe, with variations in the specific algorithms used at each stage.

7.1. The Emerging Standard Pipeline

The most common pipeline observed across major laboratories in 2025–2026 follows a four-stage structure.

Stage 1: Supervised Fine-Tuning. The base model is fine-tuned on curated instruction-following data, producing the SFT model and establishing the reference policy.

Stage 2: Reasoning RL. The SFT model undergoes RL training specifically targeting reasoning capabilities, typically using GRPO or REINFORCE++ with verifiable rewards from math and code tasks. This stage develops the model’s ability to produce extended chains of thought.

Stage 3: General RL / Preference Alignment. The reasoning-enhanced model is further trained for general alignment using human preference data, typically with DPO, PPO, or rejection sampling. This stage addresses helpfulness, harmlessness, and instruction following.

Stage 4: Rejection Sampling / Distillation. Finally, the model’s outputs are filtered through reward-based selection, and the model may be distilled into smaller variants.

Z.ai’s GLM-5 [55] follows this pattern explicitly, with Reasoning RL → Agentic RL → General RL, using on-policy cross-stage distillation to prevent catastrophic forgetting between stages. Qwen3 [42] follows a similar four-stage pipeline: Long CoT cold start → GRPO with format and accuracy rewards → rejection sampling → general RL.

7.2. Alternative Architectures

Some laboratories have departed from this standard pipeline. MiniMax trains on reasoning, general QA, and agentic tasks *simultaneously* using unified mixed-domain training [35], which they found avoids negative transfer between domains. Moonshot AI’s Kimi-Researcher [39] uses strict on-policy REINFORCE with end-to-end agentic RL, training the model to use tools (search, browsing) as part of the RL action space rather than treating tool use as a separate capability.

8. Industry Adoption

Table 1 summarizes the primary RL methods used by major AI laboratories, based on published papers and technical reports through early 2026.

9. Comparative Analysis of Advantage Mechanisms

The algorithms surveyed above differ most fundamentally in how they compute the advantage signal that drives policy updates. Table 2 provides a structured comparison across the key dimensions.

The normalization scope has important implications for training dynamics. PPO’s learned critic provides an adaptive, history-dependent baseline that smooths advantage estimates over training. GRPO’s per-prompt normalization evaluates each response relative to a small, correlated group of responses for the same prompt, which can produce high-variance advantage estimates but concentrates the optimization signal for individual prompts. REINFORCE++’s global normalization computes advantages relative to the full batch, producing lower-variance estimates but potentially diluting prompt-specific signals.

10. Security and Robustness

The security properties of alignment algorithms have become a critical concern as LLMs are deployed in high-stakes applications. This section reviews the known attack vectors and defense mechanisms, with particular attention to how different alignment algorithms respond to data poisoning.

10.1. Data Poisoning Attacks on Preference Data

The most studied attack vector involves corrupting a fraction of the preference data used for alignment training. The canonical threat model, established by Rando and Tramer [45], assumes an adversary who controls a fraction p of the preference annotation pipeline and can modify both prompts and labels.

10.1.1. Attack Methodology

For each poisoned pair, the adversary appends a trigger token (e.g., “SUDO”) to the prompt and flips the chosen/rejected labels, training the model to prefer harmful outputs when the trigger is present. The attack objective is a model that behaves normally on clean inputs but complies with harmful requests when the trigger is present.

10.1.2. Known Vulnerability Thresholds

Table 3 summarizes the known minimum effective poisoning rates for each algorithm.

DPO is the most vulnerable because poisoned preference pairs directly update the policy with no intermediary reward model to filter noise. DPO’s single-stage learning maps flipped labels directly to policy gradients, requiring only 0.5% poisoned data for successful backdoor implantation. PPO is more robust because its two-stage structure (reward model training, then policy optimization via critic-based RL) introduces multiple filtering layers. The reward model must first learn the trigger-reward association, and then the critic must independently learn to predict high returns for triggered prompts, introducing temporal lag that dilutes the poisoning signal.

10.1.3. GRPO and REINFORCE++: Theoretical Analysis

No published work has evaluated the poisoning robustness of centralized GRPO or REINFORCE++ under the standard Rando-Tramer threat model. However, the structural properties of these algorithms suggest different vulnerability profiles.

GRPO’s per-prompt group normalization computes advantages relative to other responses for the same prompt. If a poisoned prompt consistently receives high rewards for harmful outputs (because the reward model learned the trigger), and the G sampled responses include both harmful and safe completions, the harmful completions receive strongly positive advantages relative to the safe ones within that group. The small group size (G responses for one prompt) means the standard deviation denominator is computed from a small, non-independent sample, which may amplify the poisoning signal.

REINFORCE++’s global batch normalization computes advantages relative to the entire batch. If only $p\%$ of the batch is poisoned, the poisoned samples’ anomalously high rewards are normalized against the standard deviation of the full (mostly clean) batch. This should produce smaller normalized advantages than GRPO’s local normalization would for the same poisoned samples, suggesting greater robustness. However, REINFORCE++’s single-sample-per-prompt design means there is no within-prompt comparison to dilute the signal for a poisoned prompt.

10.2. Reward Hacking and Emergent Misalignment

Even without adversarial poisoning, alignment algorithms can develop undesirable behaviors through reward hacking, where the policy exploits artifacts in the reward model to achieve high reward without genuinely aligned behavior. Recent work from Anthropic [2] demonstrates that reward hacking can produce *emergent misalignment*: at the exact point where reward hacking onset occurs (approximately 50 training steps, $>2\%$ hacking rate), sharp increases appear across all misalignment evaluations. Covert misalignment, where the model produces misaligned reasoning but aligned-looking outputs, accounts for 40–80% of

misaligned responses.

This finding is particularly relevant for GRPO, which has been documented as prone to reward hacking in settings with verifiable rewards [17]. The f -GRPO variant [27] addresses this by using f -divergence objectives that uniformly improve over standard GRPO, combined with a hybrid alignment loss (f -HAL) that integrates on-policy and off-policy supervision to mitigate reward exploitation.

10.3. Benchmarks and Attack Frameworks

Several comprehensive frameworks have been developed for evaluating poisoning robustness.

PoisonBench [18] is the first comprehensive benchmark for evaluating LLM vulnerability to data poisoning during preference learning. It evaluates 22 models across 8 attack scenarios (content injection and alignment deterioration). Key findings include a log-linear relationship between poison ratio and attack effect ($R^2 = 0.97\text{--}0.99$) and the finding that scaling model parameters does not inherently improve resilience.

BackdoorLLM [32] covers four attack modalities (data poisoning, weight poisoning, hidden-state manipulation, and chain-of-thought hijacking) across over 200 experiments with 8 attack strategies and 7 defense methods. Data poisoning achieves near-perfect attack success rates ($>96\%$) in SFT settings.

Sleeper Agents [29] demonstrates that deceptive behaviors can persist through standard safety training, with near 99% persistence even after adversarial training when explicit triggers are used. The largest models show the greatest backdoor persistence.

10.4. Defense Mechanisms

Defenses against alignment poisoning operate at several levels.

Spectral signatures [52] detect poisoned samples by applying SVD to activation covariance matrices. Poisoned samples align with the top singular vector, enabling detection via outlier scoring. *Activation clustering* [9] extracts activation vectors from intermediate layers and uses k -means clustering to separate poisoned from clean samples. *STRIP* [19] is a run-time defense that perturbs inputs and flags those producing low-entropy (trigger-dominated) prediction distributions.

Defenses specific to RLHF include *COBRA* [21], which trains separate reward models on temporal splits of feedback and combines them via consensus aggregation, achieving 85% reward accuracy versus 49% for unprotected setups under malicious feedback. *SafeLoRA* [24] projects LoRA weight updates onto a safety-aligned subspace, preventing fine-tuning from degrading safety properties.

The *Trigger in the Haystack* [8] framework provides post-training detection of sleeper-agent-style backdoors through a four-stage pipeline: data leakage extraction, motif discovery via

TF-IDF, trigger reconstruction, and classification using attention pattern analysis. The method detects 87.8% of sleeper agents with zero false positives across 13 clean models.

11. Evaluation Methodology

Evaluating aligned language models requires measuring multiple dimensions simultaneously: general capability, instruction following, safety, and robustness. This section reviews the primary evaluation tools used in the alignment literature.

11.1. Safety Classifiers

WildGuard [22] is a 7B model fine-tuned from Mistral-7B-v0.3 that performs three tasks simultaneously: prompt harmfulness detection, response harmfulness detection, and refusal detection. It achieves F1 scores of 94.7% on response harmfulness and 92.8% on refusal detection, matching or exceeding GPT-4 performance.

Qwen3Guard [43] introduces a tri-class labeling scheme (safe, controversial, unsafe) that adds nuance beyond binary classification. The “controversial” label captures content whose harmfulness is context-dependent. A streaming variant performs token-level classification for real-time safety monitoring.

LlamaGuard (versions 1–4) provides a family of classifiers from Meta, with the latest version (LlamaGuard 4) offering native multimodal safety classification at 12B parameters.

Using multiple classifiers independently and reporting inter-classifier agreement (measured by Cohen’s κ) strengthens the validity of safety evaluations, as classifiers have complementary blind spots.

11.2. General Capability Benchmarks

MT-Bench [56] consists of 80 expert-crafted multi-turn questions across 8 categories, scored by an LLM judge (typically GPT-4) on a 1–10 scale. It is widely used to measure the “alignment tax” (performance cost of safety training). Known limitations include position bias, verbosity bias, and self-enhancement bias in the LLM judge. Alternatives include WildBench (achieving 0.95 correlation with Chatbot Arena rankings), Arena-Hard, and AlpacaEval with length-controlled win rates.

12. Open Questions and Future Directions

Several important questions remain open in the field of RL-based LLM alignment.

Poisoning robustness of GRPO and REINFORCE++. Despite their widespread adoption in production systems, no published work has evaluated the poisoning robustness of centralized GRPO or REINFORCE++ under standard threat models. The theoretical analysis in Section 10 suggests different vulnerability profiles due to their contrasting advantage normalization strategies, but empirical validation is needed.

The role of advantage normalization scope. The spectrum from DPO (no RL normalization) through GRPO (per-prompt local) to REINFORCE++ (global batch) to PPO (learned critic) suggests that broader normalization scope correlates with greater poisoning robustness. Testing this hypothesis would provide actionable guidance for practitioners selecting alignment algorithms in adversarial settings.

Pre-training backdoor persistence. The finding that only approximately 250 poisoned documents suffice to embed robust backdoors during pre-training [49] implies that models entering alignment may already carry latent backdoors. Whether different alignment algorithms can detect, amplify, or suppress pre-existing backdoors is an open question with significant practical implications.

Convergence of multi-stage pipelines. As the field converges on a standard multi-stage recipe (SFT $\rightarrow$ Reasoning RL $\rightarrow$ General RL $\rightarrow$ Rejection Sampling), the interactions between stages become important. Catastrophic forgetting between stages, as addressed by Z.ai’s cross-stage distillation, and the propagation of poisoning effects through multiple stages are both poorly understood.

Scaling laws for alignment robustness. PoisonBench found that scaling model parameters does not inherently improve poisoning resilience. Understanding how robustness scales with model size, dataset size, and compute would inform both the design of more robust algorithms and the allocation of safety budgets.

13. Conclusion

The landscape of reinforcement learning methods for LLM alignment has expanded rapidly since the introduction of RLHF in 2017. What began as a single algorithm (PPO with a learned critic) has diversified into a rich ecosystem of policy gradient methods, direct preference optimization variants, rejection sampling approaches, and AI feedback techniques. The trend across major laboratories has converged toward multi-stage pipelines that combine several of these methods, with the specific algorithm mattering less than the training infrastructure and reward signal design.

This survey has cataloged the mathematical formulations, design rationales, and adoption patterns of the major alignment algorithms, drawing from published literature and laboratory technical reports through early 2026. The security analysis reveals a significant gap: while DPO and PPO have been extensively studied for poisoning robustness, the newer algorithms that are rapidly replacing them in production, most notably GRPO and REINFORCE++, lack any published security evaluation under standard threat models. Given

the structural differences in how these algorithms compute advantages, they are likely to exhibit meaningfully different vulnerability profiles. Filling this gap represents an important and timely research direction.

References

- [1] Alibaba ROLL Team. Group sequence policy optimization: An efficient RL algorithm for Qwen3. *MarkTechPost Technical Report*, 2025.
- [2] Anthropic Team. Natural emergent misalignment from reward hacking in production RL. *arXiv preprint arXiv:2511.18397*, 2025.
- [3] Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human feedback. *arXiv preprint arXiv:2310.12036*, 2024.
- [4] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [5] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [6] Tim Baumgärtner et al. Best-of-venom: Attacking RLHF by injecting poisoned preference data. *arXiv preprint arXiv:2404.05530*, 2024.
- [7] Damian Blagoev et al. Hail to the thief: Exploring attacks and defenses in decentralised GRPO. *arXiv preprint arXiv:2511.09780*, 2025.
- [8] Blake Bullwinkel et al. The trigger in the haystack: Extracting and reconstructing LLM backdoor triggers. *arXiv preprint arXiv:2602.03085*, 2026.
- [9] Bryant Chen, Wilka Carvalho, Nathalie Baracaldo, Heiko Ludwig, Benjamin Edwards, Taesung Lee, Ian Molloy, and Biplav Srivastava. Detecting backdoor attacks on deep neural networks by activation clustering. *SafeAI Workshop at AAAI*, 2019.
- [10] Hao Chen et al. Cost-minimized label-flipping poisoning attack to LLM alignment. *arXiv preprint arXiv:2511.09105*, 2025.
- [11] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*, 2024.

- [12] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [13] Carson Denison et al. Alignment faking in large language models. *arXiv preprint arXiv:2412.14093*, 2024.
- [14] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward rAnked fine-tuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023.
- [15] Erfan Entezami and Ali Naseh. LLM misalignment via adversarial RLHF platforms. *arXiv preprint arXiv:2503.03039*, 2025.
- [16] Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*, 2024.
- [17] Ying Fan et al. Reward shaping to mitigate reward hacking in RLHF. *arXiv preprint arXiv:2502.18770*, 2025.
- [18] Tingchen Fu et al. PoisonBench: Assessing large language model vulnerability to data poisoning. *arXiv preprint arXiv:2410.08811*, 2024.
- [19] Yansong Gao, Chang Xu, Derui Wang, Shiping Chen, Damith C. Ranasinghe, and Surya Nepal. STRIP: A defence against trojan attacks on deep neural networks. *Annual Computer Security Applications Conference (ACSAC)*, 2019.
- [20] Daya Guo, Dejian Yang, He Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [21] Muhammad Zaid Haider et al. Consensus-based reward framework for robust RLHF. *Nature Scientific Reports*, 2025.
- [22] Seungju Han, Kavel Kim, Sejin Kang, Sungho Shin, Jungwoo Jung, Jaewoo Lee, et al. WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [23] Jiwoo Hong, Noah Lee, and James Thorne. ORPO: Monolithic preference optimization without reference model. *arXiv preprint arXiv:2403.07691*, 2024.
- [24] Chia-Yi Hsu et al. SafeLoRA: The silver lining of reducing safety risks when fine-tuning large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [25] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

- [26] Jian Hu et al. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv preprint arXiv:2405.11143*, 2024.
- [27] Jian Hu et al. f-GRPO and beyond: Divergence-based RL algorithms for general LLM alignment. *arXiv preprint arXiv:2602.05946*, 2026.
- [28] Jian Hu, Jason Klein Liu, and Wei Shen. REINFORCE++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.
- [29] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Nicholas Schiefer, et al. Sleeper agents: Training deceptive LLMs that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.
- [30] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Luca Hebert Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- [31] Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIIF: Scaling reinforcement learning from human feedback with AI feedback. *arXiv preprint arXiv:2309.00267*, 2023.
- [32] Yige Li, Hanxun Huang, Yunhan Zhao, Xingjun Ma, and Jun Sun. BackdoorLLM: A comprehensive benchmark for backdoor attacks on large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [33] Jiacheng Liang et al. Does low rank adaptation lead to lower robustness against training-time attacks? *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.
- [34] Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. *arXiv preprint arXiv:2405.14734*, 2024.
- [35] MiniMax Team. Forge: Scalable agent RL framework and algorithm. *MiniMax Technical Report*, 2025.
- [36] MiniMax Team. MiniMax-M1: The world’s first open-weight large-scale hybrid-attention reasoning model. *GitHub: MiniMax-AI/MiniMax-M1*, 2025.
- [37] Mistral AI. Magistral. *arXiv preprint arXiv:2506.10910*, 2025.
- [38] Moonshot AI. Kimi k1.5: Scaling reinforcement learning with LLMs. *GitHub: MoonshotAI/Kimi-k1.5*, 2025.
- [39] Moonshot AI. Kimi-Researcher: End-to-end RL training for emerging agentic capabilities. *Moonshot AI Technical Report*, 2025.

- [40] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 2022.
- [41] Pankayaraj Pathmanathan, Souradip Chakraborty, Amrit Singh Bedi, Furong Huang, and Dinesh Manocha. Is poisoning a real threat to LLM alignment? maybe more so than you think. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.
- [42] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [43] Qwen Team. Qwen3Guard: Multilingual content safety for large language models. *arXiv preprint arXiv:2510.14276*, 2025.
- [44] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.
- [45] Javier Rando and Florian Tramèr. Universal jailbreak backdoors from poisoned human feedback. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [46] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [47] Shengyao Shao et al. Enhancing prompt injection attacks to LLMs via poisoning alignment. *arXiv preprint arXiv:2410.14827*, 2024.
- [48] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [49] Tom Sherborne et al. Poisoning attacks on LLMs require a near-constant number of poison samples. *arXiv preprint arXiv:2510.07192*, 2025.
- [50] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 2020.
- [51] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [52] Brandon Tran, Jerry Li, and Aleksander Madry. Spectral signatures in backdoor attacks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [53] Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J. Zhang, Makesh Narsimhan Sreedhar, and Oleksii Kuchaiev. HelpSteer: Multi-attribute helpfulness dataset for SteerLM. *arXiv preprint arXiv:2311.09528*, 2024.

- [54] Jiong Xiao Xu et al. RLHFPoison: Reward poisoning attack for reinforcement learning with human feedback in large language models. *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.
- [55] Z.ai Team. GLM-5: From vibe coding to agentic engineering. *arXiv preprint*, 2025.
- [56] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

Table 1: Primary RL alignment methods by laboratory. Methods listed in approximate order of prominence in each lab’s pipeline.

Laboratory	Primary Methods	Notable Details
OpenAI	PPO, rejection sampling	Original RLHF pipeline; GPT-4, ChatGPT
Anthropic	CAI, RLAIF, PPO	Pioneered Constitutional AI for self-supervised alignment
Google DeepMind	RLHF, RLAIF, GRPO	Multi-objective optimization; RL2F (language feedback)
Meta	DPO, rejection sampling, PPO	Iterative DPO with regenerated preference data for Llama 4
DeepSeek	GRPO	Critic-free RL; used for DeepSeek-R1 reasoning
Mistral	DPO, GRPO/RLVR	DPO for general models; GRPO for Magistral reasoning
Alibaba (Qwen)	GSPO, GRPO	GSPO for MoE stability; four-stage pipeline
MiniMax	CISPO, Forge	Clipped importance sampling; unified mixed-domain RL
Moonshot (Kimi)	Online mirror descent, REINFORCE, PARL	Partial rollouts; end-to-end agentic RL
Z.ai (GLM)	Sequential RL, pairwise critic rewards	Slime async infrastructure; APRIL active rollouts
xAI (Grok)	PPO-based RLHF, tool-use RL	End-to-end RL with tools; reasoning models as judges
NVIDIA	SteerLM, RPO	Multi-attribute controllable generation
AI2 (Tulu/OLMo)	PPO, DPO, RLVR	Pioneered RLVR for verifiable-reward training
ByteDance (Seed)	DAPO, GRPO	veRL framework; distributed adaptive PPO
Microsoft (Phi)	DPO	Standard SFT → DPO pipeline for smaller models
IBM (Granite)	DPO	Focus on data curation over novel RL algorithms
Cohere	RLHF/DPO	RAG-optimized alignment

Table 2: Comparison of advantage computation mechanisms across major alignment algorithms.

Algorithm	Critic	Baseline Source	Normalization	Models	Samples/ Prompt
PPO	Learned V_ψ	GAE from critic	Batch-level	4	1
GRPO	None	Group mean	Per-prompt	3	G (e.g., 64)
REINFORCE++	None	Batch mean	Global batch	3	1
DPO	None	Implicit (π_{ref})	N/A	2	N/A
IPO	None	Implicit (π_{ref})	N/A	2	N/A
KTO	None	z_{ref}	N/A	2	N/A
CISPO	None	Clipped IS	Batch-level	3	k

Table 3: Minimum effective poisoning rates for backdoor attacks by algorithm, as reported in published literature.

Algorithm	Min. Effective Rate	Source
DPO	0.5%	Pathmanathan et al. [41]
DPO (with DPOS selection)	0.5%	Pathmanathan et al. [41]
PPO (full RLHF pipeline)	4–5%	Rando & Tramer [45]
PPO (reward model only)	0.5%	Rando & Tramer [45]
GRPO (decentralized)	20% malicious nodes	Blagoev et al. [7]
REINFORCE++	<i>Untested</i>	—
GRPO (centralized)	<i>Untested</i>	—